Лабораторная работа № 2 по МВМ

МНК с помощью QR- и SVD-разложений

Выполнил: студент группы А–14–23 Кузнецов Давид
Вариант 9 (нечётный)

Москва, 2026

1. Постановка задачи

Рассматривается переопределённая система

$$Ax = b, \tag{1}$$

где матрица A имеет $N+2$ строки и N столбцов. Требуется найти решение в смысле наименьших квадратов, то есть минимизировать величину

$$\|Ax - b\|_2 . \tag{2}$$

Работа состоит из двух частей. В первой части исследуется матрица полного столбцового ранга, во второй части — матрица неполного ранга нечётного варианта. Для обеих частей необходимо выполнить QR-разложение отражениями Хаусхолдера, проверить его матричным умножением, построить SVD и псевдообратную матрицу, а также исследовать время работы алгоритмов на больших системах.

Тестовое решение строится по году рождения Давида Гильберта — 1862:

$$x_* = (1,\ 1,\ 8,\ 6,\ 2)^T. \tag{3}$$

Первый элемент равен единице, следующие четыре элементы совпадают с цифрами года рождения. Тестовая правая часть вычисляется как $b_{\text{test}} = Ax_*$. Все числовые результаты далее приведены с шестью значащими цифрами.

2. Используемые методы

2.1. QR-разложение отражениями Хаусхолдера

На шаге k рассматривается часть x текущего столбца, расположенная начиная с диагонального элемента. В программе выбирается

$$\alpha = -\operatorname{sign}(x_1)\,\|x\|_2 , \qquad v = \frac{x - \alpha e_1}{\|x - \alpha e_1\|_2}. \tag{4}$$

Отражение Хаусхолдера имеет вид

$$H_k = I - 2vv^T. \tag{5}$$

Оно зануляет элементы столбца под диагональю. Последовательное применение отражений приводит к разложению

$$A = QR, \qquad Q = H_1 H_2 \dots H_N. \tag{6}$$

При решении системы матрица Q явно не формируется: к правой части последовательно применяются сохранённые векторы отражений,

$$c = Q^T b. \tag{7}$$

Если A имеет полный столбцовый ранг, решение МНК находится из верхнетреугольной системы

$$R_1 x = c_1. \tag{8}$$

В части 2 используется выбор ведущего столбца. На каждом шаге выбирается столбец с наибольшей нормой оставшейся части. В результате получается разложение

$$A\Pi = QR, \tag{9}$$

где Π — матрица перестановок. Если $r = \operatorname{rank} A < N$, решается система $R_{11} z_1 = c_1$, свободные компоненты z_2 принимаются равными нулю, после чего выполняется обратная перестановка $x_\pi = z$. Такое решение является одним из решений МНК.

Матрица Q для отчёта восстанавливается по столбцам: отражения применяются к векторам стандартного базиса в обратном порядке. Промежуточные матрицы H_k явно не формируются и не перемножаются.

2.2. Сингулярное разложение и псевдообратная матрица

Сингулярное разложение вычисляется функцией `numpy.linalg.svd`:

$$A = U \Sigma V^T, \tag{10}$$

где U и V — ортогональные матрицы, а на диагонали Σ расположены сингулярные числа σ_i.

Псевдообратная матрица строится по формуле

$$A^+ = V \Sigma^+ U^T, \tag{11}$$

где все сингулярные числа, превышающие численный порог, заменяются обратными, а остальные элементы остаются нулевыми. Решение

$$x_{\text{SVD}} = A^+ b \tag{12}$$

минимизирует невязку и среди всех решений МНК имеет минимальную евклидову норму.

Для проверки псевдообратной матрицы используются свойства проекторов

$$(A^+A)^2 = A^+A, \qquad (AA^+)^2 = AA^+, \tag{13}$$

$$(A^+A)^T = A^+A, \qquad (AA^+)^T = AA^+. \tag{14}$$

3. Часть 1. Полноранговая система

Элементы матрицы задаются формулами

$$A_{i,j} = \frac{10}{i+1}, \qquad i = 0, \ldots, N+1, \quad j = 0, \ldots, N-1, \tag{15}$$

причём диагональные элементы имеют приоритет:

$$A_{i,i} = \min\{N, 10\}. \tag{16}$$

Для $N = 5$ получена матрица

$$A = \begin{pmatrix} 5 & 10 & 10 & 10 & 10 \\ 5 & 5 & 5 & 5 & 5 \\ 3.33333 & 3.33333 & 5 & 3.33333 & 3.33333 \\ 2.5 & 2.5 & 2.5 & 5 & 2.5 \\ 2 & 2 & 2 & 2 & 5 \\ 1.66667 & 1.66667 & 1.66667 & 1.66667 & 1.66667 \\ 1.42857 & 1.42857 & 1.42857 & 1.42857 & 1.42857 \end{pmatrix}. \tag{17}$$

Её ранг равен количеству столбцов:

$$\operatorname{rank} A = 5. \tag{18}$$

Для тестового решения вычислена правая часть

$$b_{\text{test}} = (175,\ 90,\ 73.3333,\ 60,\ 42,\ 30,\ 25.7143)^T. \tag{19}$$

При изменении первого элемента получаем

$$\widetilde{b} = (176,\ 90,\ 73.3333,\ 60,\ 42,\ 30,\ 25.7143)^T. \tag{20}$$

Однако $\operatorname{rank}[A \mid \widetilde{b}] = 5$, поэтому система остаётся совместной. Это связано со структурой конкретной матрицы: вектор e_1 принадлежит пространству её столбцов, поскольку

$$A(-0.2,\ 0.2,\ 0,\ 0,\ 0)^T = e_1. \tag{21}$$

Поэтому для построения несовместной основной задачи изменён последний элемент правой части:

$$b_{\text{main}} = (175,\ 90,\ 73.3333,\ 60,\ 42,\ 30,\ 26.7143)^T. \tag{22}$$

При этом

$$\operatorname{rank}[A \mid b_{\text{main}}] = 6 > \operatorname{rank} A, \tag{23}$$

то есть точного решения не существует.

Раздел A. Решение основной задачи и невязка

После QR-разложения получена матрица

$$R = \begin{pmatrix} -8.72810 & -11.5924 & -12.2289 & -12.3085 & -12.2798 \\ 0 & 4.09826 & 3.65339 & 3.59778 & 3.61780 \\ 0 & 0 & -1.47469 & 0.460053 & 0.441651 \\ 0 & 0 & 0 & -2.29676 & 0.407485 \\ 0 & 0 & 0 & 0 & -2.81700 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}, \tag{24}$$

а преобразованная правая часть равна

$$Q^T b_{\text{main}} = (-216.726,\ 62.0333,\ -8.04878,\ -12.8686,\ -5.54403,\ -0.0416820,\ 0.964273)^T. \tag{25}$$

Решение основной задачи:

$$\boxed{x_{\mathrm{QR}} = (1.09582,\ 1.12776,\ 7.90418,\ 5.95209,\ 1.96806)^T.} \tag{26}$$

Абсолютная и относительная невязки:

$$\|Ax_{\mathrm{QR}} - b_{\mathsf{main}}\|_2 = 0.965173, \tag{27}$$

$$\frac{\|Ax_{\mathrm{QR}} - b_{\mathsf{main}}\|_2}{\|b_{\mathsf{main}}\|_2} = 0.00427049. \tag{28}$$

Раздел Б. Тестовый пример

Для тестовой задачи верхняя треугольная система имеет вид

$$R_1 x = c_1, \tag{29}$$

где

$$R_1 = \begin{pmatrix} -8.72810 & -11.5924 & -12.2289 & -12.3085 & -12.2798 \\ 0 & 4.09826 & 3.65339 & 3.59778 & 3.61780 \\ 0 & 0 & -1.47469 & 0.460053 & 0.441651 \\ 0 & 0 & 0 & -2.29676 & 0.407485 \\ 0 & 0 & 0 & 0 & -2.81700 \end{pmatrix}, \tag{30}$$

$$c_1 = (-216.563,\ 62.1477,\ -8.15393,\ -12.9656,\ -5.63400)^T. \tag{31}$$

Метод возвращает

$$x_{\mathrm{QR,test}} = (1,\ 1,\ 8,\ 6,\ 2)^T. \tag{32}$$

Погрешность решения и невязка равны

$$\|x_{\mathrm{QR,test}} - x_*\|_2 = 1.16927 \cdot 10^{-14}, \tag{33}$$

$$\|Ax_{\mathrm{QR,test}} - b_{\mathsf{test}}\|_2 = 3.75984 \cdot 10^{-14}. \tag{34}$$

Следовательно, тестовое решение воспроизведено с точностью машинного округления.

Раздел В. Матричное QR-разложение

Восстановленная ортогональная матрица имеет вид

$$Q = \begin{pmatrix} -0.572862 & 0.819652 & 0 & 0 & 0 & 0 & 0 \\ -0.572862 & -0.400379 & 0.368042 & 0.339571 & 0.314904 & -0.305215 & -0.261612 \\ -0.381908 & -0.266919 & -0.884816 & 0 & 0 & 0 & 0 \\ -0.286431 & -0.200190 & 0.184021 & -0.918704 & 0 & 0 & 0 \\ -0.229145 & -0.160152 & 0.147217 & 0.135828 & -0.939000 & 0 & 0 \\ -0.190954 & -0.133460 & 0.122681 & 0.113190 & 0.104968 & 0.951371 & -0.0416820 \\ -0.163675 & -0.114394 & 0.105155 & 0.0970202 & 0.0899727 & -0.0416820 & 0.964273 \end{pmatrix}. \tag{35}$$

В этой части перестановки столбцов не выполнялись, поэтому $\Pi = I_5$. Проверки дали

$$\left\|Q^T Q - I_7\right\|_F = 1.46209 \cdot 10^{-15}, \tag{36}$$

$$\|A - QR\|_F = 9.94006 \cdot 10^{-15}. \tag{37}$$

Таким образом, равенство $A = QR$ выполняется с точностью машинного округления.

Раздел Г. SVD и псевдообратная матрица

Полное SVD содержит квадратные матрицы $U \in \mathbb{R}^{7\times 7}$ и $V \in \mathbb{R}^{5\times 5}$:

$$U = \begin{pmatrix} -0.768677 & 0.0974833 & -0.609589 & -0.128784 & -0.107000 & 0 & 0 \\ -0.413701 & -0.142857 & 0.594733 & -0.00415869 & -0.541407 & -0.305215 & -0.261612 \\ -0.305521 & -0.185928 & 0.352019 & -0.542894 & 0.673376 & 0 & 0 \\ -0.251853 & -0.507799 & 0.0181580 & 0.749905 & 0.340622 & 0 & 0 \\ -0.219605 & 0.820808 & 0.286831 & 0.355391 & 0.263577 & 0 & 0 \\ -0.137900 & -0.0476191 & 0.198244 & -0.00138623 & -0.180469 & 0.951371 & -0.0416820 \\ -0.118200 & -0.0408164 & 0.169924 & -0.00118820 & -0.154688 & -0.0416820 & 0.964273 \end{pmatrix}, \tag{38}$$

$$V = \begin{pmatrix} -0.314058 & -0.216605 & 0.907990 & 0.0508151 & -0.165594 \\ -0.457791 & -0.0441466 & -0.299824 & -0.254491 & -0.796127 \\ -0.476834 & -0.153789 & -0.0673320 & -0.683501 & 0.526565 \\ -0.481338 & -0.493323 & -0.281835 & 0.634403 & 0.207482 \\ -0.482429 & 0.827112 & 0.0411654 & 0.251021 & 0.135798 \end{pmatrix}. \tag{39}$$

Сингулярные числа:

$$(26.7397,\ 2.82628,\ 2.52352,\ 2.10910,\ 0.848492). \tag{40}$$

Следовательно, $\operatorname{rank} A = 5$. Матрица Σ имеет вид

$$\Sigma = \begin{pmatrix} 26.7397 & 0 & 0 & 0 & 0 \\ 0 & 2.82628 & 0 & 0 & 0 \\ 0 & 0 & 2.52352 & 0 & 0 \\ 0 & 0 & 0 & 2.10910 & 0 \\ 0 & 0 & 0 & 0 & 0.848492 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}. \tag{41}$$

Численные проверки:

$$\left\| U^T U - I_7 \right\|_F = 2.25710 \cdot 10^{-15}, \tag{42}$$

$$\left\| V^T V - I_5 \right\|_F = 1.57211 \cdot 10^{-15}, \tag{43}$$

$$\left\| U^T A V - \Sigma \right\|_F = 2.81514 \cdot 10^{-14}, \tag{44}$$

$$\left\| A - U \Sigma V^T \right\|_F = 2.98755 \cdot 10^{-14}. \tag{45}$$

SVD даёт для основной задачи

$$x_{\mathrm{SVD}} = (1.09582,\ 1.12776,\ 7.90418,\ 5.95209,\ 1.96806)^T, \tag{46}$$

$$\left\| x_{\mathrm{QR}} - x_{\mathrm{SVD}} \right\|_2 = 2.41886 \cdot 10^{-14}. \tag{47}$$

Для тестовой задачи ошибка SVD-решения равна $1.17788 \cdot 10^{-14}$.

Псевдообратная матрица получена непосредственно из найденного SVD. Сначала задавался численный порог

$$\tau = \varepsilon_{\mathrm{mach}} \max\{m, n\} \sigma_1. \tag{48}$$

Затем для каждого сингулярного числа вычислялось

$$\sigma_i^+ = \begin{cases} 1/\sigma_i, & \sigma_i > \tau, \\ 0, & \sigma_i \le \tau. \end{cases} \tag{49}$$

Полученные значения помещались на диагональ прямоугольной матрицы $\Sigma^+ \in \mathbb{R}^{5\times 7}$, после чего выполнялось умножение

$$A^+ = V\Sigma^+ U^T. \tag{50}$$

В полноранговой части обратными заменены все пять сингулярных чисел. В результате получена матрица

Псевдообратная матрица:

$$A^+ = \begin{pmatrix} -0.2 & 0.335361 & 0 & 0 & 0 & 0.111787 & 0.0958175 \\ 0.2 & 0.447148 & -0.6 & -0.4 & -0.333333 & 0.149049 & 0.127757 \\ 0 & -0.335361 & 0.6 & 0 & 0 & -0.111787 & -0.0958175 \\ 0 & -0.167681 & 0 & 0.4 & 0 & -0.0558935 & -0.0479087 \\ 0 & -0.111787 & 0 & 0 & 0.333333 & -0.0372624 & -0.0319392 \end{pmatrix}. \tag{51}$$

Для полноранговой прямоугольной матрицы $A^+A \approx I_5$, а AA^+ является ортогональным проектором в $\mathbb{R}^7$ и не должно совпадать с I_7. Получены значения

$$\left\|A^+A - I_5\right\|_F = 3.47842 \cdot 10^{-15}, \tag{52}$$

$$\left\|AA^+ - I_7\right\|_F = 1.41421, \tag{53}$$

$$\left\|(A^+A)^2 - A^+A\right\|_F = 3.47842 \cdot 10^{-15}, \tag{54}$$

$$\left\|(AA^+)^2 - AA^+\right\|_F = 1.03335 \cdot 10^{-14}, \tag{55}$$

$$\left\|(A^+A)^T - A^+A\right\|_F = 4.46612 \cdot 10^{-15}, \tag{56}$$

$$\left\|(AA^+)^T - AA^+\right\|_F = 1.45194 \cdot 10^{-14}. \tag{57}$$

Решение A^+b_{main} совпало с SVD-решением, причём

$$\left\|A^+b_{\text{main}} - x_{\text{SVD}}\right\|_2 = 1.22528 \cdot 10^{-14}. \tag{58}$$

Раздел Д. Большая система и анализ трудоёмкости

Для каждого размера время измерялось три раза, после чего использовалась медиана. Экспериментальный показатель роста вычислялся по формуле

$$p = \frac{\ln(T_2/T_1)}{\ln(N_2/N_1)}. \tag{59}$$

Таблица 1: Результаты для полноранговой системы

N	T_{QR}, с	p_{QR}	T_{SVD}, с	p_{SVD}	$\|r_{QR}\|_2$	$\|r_{SVD}\|_2$
100	0.0213613	–	0.175287	–	0.999952	0.999952
500	0.250057	1.52855	0.107866	-0.301678	0.999998	0.999998
1000	2.02383	3.01676	0.478178	2.14831	1.00000	1.00000
1500	6.76080	2.97473	1.42960	2.70101	1.00000	1.00000
2000	15.5329	2.89146	3.85456	3.44777	1.00000	1.00000

Таблица 2: Относительные невязки для полноранговой системы

N	$\|r_{QR}\|_2 / \|b\|_2$	$\|r_{SVD}\|_2 / \|b\|_2$
100	$1.79841 \cdot 10^{-4}$	$1.79841 \cdot 10^{-4}$
500	$3.64722 \cdot 10^{-5}$	$3.64722 \cdot 10^{-5}$
1000	$1.82830 \cdot 10^{-5}$	$1.82830 \cdot 10^{-5}$
1500	$1.22006 \cdot 10^{-5}$	$1.22006 \cdot 10^{-5}$
2000	$9.15523 \cdot 10^{-6}$	$9.15523 \cdot 10^{-6}$

Для плотной матрицы размера $(N+2) \times N$ QR-разложение и полное SVD требуют $O(N^3)$ арифметических операций и $O(N^2)$ памяти. При больших N значения p_{QR} близки к 3; значения p_{SVD} также приближаются к кубическому порядку. Полученное значение $p_{SVD} = 3.44777$ при $N = 2000$ несколько превышает 3 из-за влияния памяти, кэша и особенностей оптимизированной реализации SVD в библиотеке NumPy. Результат согласуется с теоретической трудоёмкостью $O(N^3)$. Отрицательное значение p_{SVD} между $N = 100$ и $N = 500$ связано с малой длительностью первых запусков, накладными расходами и работой оптимизированных библиотечных процедур. Оно не характеризует асимптотику алгоритма.

Расчёты выполнены до $N = 2000$. Ограничение в один час фактически не было достигнуто: на этом размере максимальное измеренное время равно 15.5329 с. Дальнейшее увеличение N ограничивается памятью, поскольку SVD одновременно хранит несколько плотных матриц размера порядка N^2.

4. Часть 2. Система неполного ранга

Для нечётного варианта элементы матрицы задаются формулой

$$A_{i,j} = i + j - 2ij, \qquad i = 1, \ldots, N+2, \quad j = 1, \ldots, N. \tag{60}$$

Каждый столбец можно представить как

$$A_{:,j} = (1 - 2j)(1, 2, \ldots, N+2)^T + j(1, 1, \ldots, 1)^T. \tag{61}$$

Следовательно, все столбцы лежат в линейной оболочке двух векторов и $\operatorname{rank} A \le 2$. Первые два столбца линейно независимы, поэтому при $N \ge 2$

$$\operatorname{rank} A = 2. \tag{62}$$

При $N = 5$ получена матрица

$$A = \begin{pmatrix} 0 & -1 & -2 & -3 & -4 \\ -1 & -4 & -7 & -10 & -13 \\ -2 & -7 & -12 & -17 & -22 \\ -3 & -10 & -17 & -24 & -31 \\ -4 & -13 & -22 & -31 & -40 \\ -5 & -16 & -27 & -38 & -49 \\ -6 & -19 & -32 & -45 & -58 \end{pmatrix}. \tag{63}$$

Тестовая и основная правые части:

$$b_{\text{test}} = (-43,\ -147,\ -251,\ -355,\ -459,\ -563,\ -667)^T, \tag{64}$$

$$b_{\text{main}} = (-42,\ -147,\ -251,\ -355,\ -459,\ -563,\ -667)^T. \tag{65}$$

Проверка рангов дала

$$\operatorname{rank} A = 2, \qquad \operatorname{rank}[A \mid b_{\text{test}}] = 2, \qquad \operatorname{rank}[A \mid b_{\text{main}}] = 3. \tag{66}$$

Тестовая система совместна, а основная система несовместна.

Раздел А. Решение основной задачи и невязка

QR-разложение с выбором ведущего столбца определило перестановку

$$\pi = (5,\ 1,\ 3,\ 4,\ 2) \tag{67}$$

и ранг $r = 2$. Матрица R:

$$R = \begin{pmatrix} 94.8420 & 9.52110 & 52.1815 & 73.5118 & 30.8513 \\ 0 & -0.590456 & -0.295228 & -0.147614 & -0.442842 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}. \tag{68}$$

Преобразованная правая часть:

$$Q^T b_{\text{main}} = (1088.54,\ -4.96088,\ -0.308762,\ -0.317917,\ -0.327071,\ -0.336225,\ -0.345379)^T. \tag{69}$$

При выборе свободных переменных равными нулю получено

$$z = (10.6339,\ 8.40179,\ 0,\ 0,\ 0)^T, \tag{70}$$

$$\boxed{x_{\text{QR}} = (8.40179,\ 0,\ 0,\ 0,\ 10.6339)^T.} \tag{71}$$

Невязки:

$$\|Ax_{\text{QR}} - b_{\text{main}}\|_2 = 0.731925, \tag{72}$$

$$\frac{\|Ax_{\text{QR}} - b_{\text{main}}\|_2}{\|b_{\text{main}}\|_2} = 0.000672386. \tag{73}$$

Раздел Б. Тестовый пример

Главная треугольная система имеет вид

$$\begin{pmatrix} 94.8420 & 9.52110 \\ 0 & -0.590456 \end{pmatrix} \begin{pmatrix} z_1 \\ z_2 \end{pmatrix} = \begin{pmatrix} 1088.58 \\ -4.28081 \end{pmatrix}. \tag{74}$$

После обратной перестановки найдено решение

$$x_{\text{QR,test}} = (7.25,\ 0,\ 0,\ 0,\ 10.75)^T. \tag{75}$$

Его невязка равна

$$\|Ax_{\text{QR,test}} - b_{\text{test}}\|_2 = 1.83232 \cdot 10^{-13}. \tag{76}$$

При этом

$$\|x_{\text{QR,test}} - x_*\|_2 = 14.7182. \tag{77}$$

Большое отличие векторов решения не является ошибкой: при $\operatorname{rank} A < N$ совместная система имеет бесконечно много решений. Оба вектора удовлетворяют системе с точностью машинного округления.

Раздел В. Матричное QR-разложение

Построены матрицы

$$Q = \begin{pmatrix} -0.0421754 & -0.680079 & -0.308762 & -0.317917 & -0.327071 & -0.336225 & -0.345379 \\ -0.137070 & -0.516649 & -0.242458 & -0.0183084 & 0.205842 & 0.429992 & 0.654141 \\ -0.231965 & -0.353219 & 0.902729 & -0.0674368 & -0.0376025 & -0.00776830 & 0.0220659 \\ -0.326859 & -0.189789 & -0.0935134 & 0.899904 & -0.106678 & -0.113261 & -0.119843 \\ -0.421754 & -0.0263596 & -0.0897559 & -0.132755 & 0.824246 & -0.218753 & -0.261752 \\ -0.516649 & 0.137070 & -0.0859983 & -0.165414 & -0.244830 & 0.675754 & -0.403662 \\ -0.611544 & 0.300500 & -0.0822407 & -0.198073 & -0.313906 & -0.429738 & 0.454429 \end{pmatrix}, \tag{78}$$

$$\Pi = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{pmatrix}. \tag{79}$$

Непосредственная проверка дала

$$\left\|Q^T Q - I_7\right\|_F = 7.25885 \cdot 10^{-16}, \tag{80}$$

$$\|A\Pi - QR\|_F = 2.97078 \cdot 10^{-14}. \tag{81}$$

Следовательно, равенство $A\Pi = QR$ выполнено с точностью машинного округления.

Раздел Г. SVD и псевдообратная матрица

Получены квадратные ортогональные матрицы

$$U = \begin{pmatrix} 0.0404702 & -0.680182 & 0.731375 & -0.0100256 & -0.0222539 & 0.00623714 & 0.0130211 \\ 0.135774 & -0.516991 & -0.490452 & 0.427508 & -0.411123 & 0.139284 & -0.320245 \\ 0.231078 & -0.353800 & -0.332529 & -0.0699239 & 0.660527 & 0.345751 & 0.387459 \\ 0.326383 & -0.190608 & -0.179753 & -0.301557 & 0.246333 & -0.753353 & -0.325076 \\ 0.421687 & -0.0274170 & -0.0781470 & -0.193253 & -0.527616 & -0.181644 & 0.683082 \\ 0.516991 & 0.135774 & 0.0896762 & -0.506514 & -0.136736 & 0.506499 & -0.417672 \\ 0.612295 & 0.298966 & 0.259829 & 0.653765 & 0.190869 & -0.0627742 & -0.0205690 \end{pmatrix}, \tag{82}$$

$$V = \begin{pmatrix} -0.0706549 & -0.771368 & 0.213174 & 0.595188 & 0.0175556 \\ -0.228916 & -0.497592 & -0.154414 & -0.600172 & -0.562094 \\ -0.387178 & -0.223815 & -0.566790 & -0.152929 & 0.674805 \\ -0.545439 & 0.0499605 & 0.744125 & -0.274377 & 0.266450 \\ -0.703701 & 0.323737 & -0.236095 & 0.432291 & -0.396716 \end{pmatrix}. \tag{83}$$

Сингулярные числа:

$$(134.776,\ 0.734517,\ 4.55740 \cdot 10^{-15},\ 1.51721 \cdot 10^{-16},\ 1.39646 \cdot 10^{-16}). \tag{84}$$

После применения численного порога последние три числа считаются нулевыми, поэтому $\operatorname{rank} A = 2$. Матрица Σ:

$$\Sigma = \begin{pmatrix} 134.776 & 0 & 0 & 0 & 0 \\ 0 & 0.734517 & 0 & 0 & 0 \\ 0 & 0 & 4.55740 \cdot 10^{-15} & 0 & 0 \\ 0 & 0 & 0 & 1.51721 \cdot 10^{-16} & 0 \\ 0 & 0 & 0 & 0 & 1.39646 \cdot 10^{-16} \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}. \tag{85}$$

Проверки дали

$$\left\|U^T U - I_7\right\|_F = 1.11422 \cdot 10^{-15}, \tag{86}$$

$$\left\|V^T V - I_5\right\|_F = 5.63331 \cdot 10^{-16}, \tag{87}$$

$$\left\|U^T A V - \Sigma\right\|_F = 2.20746 \cdot 10^{-14}, \tag{88}$$

$$\left\|A - U\Sigma V^T\right\|_F = 3.40214 \cdot 10^{-14}. \tag{89}$$

Для тестовой задачи SVD возвращает решение минимальной нормы

$$x_{\text{SVD,test}} = (2.2,\ 2.9,\ 3.6,\ 4.3,\ 5)^T, \tag{90}$$

имеющее невязку $1.63888 \cdot 10^{-13}$. Для основной задачи получено

$$\boxed{x_{\text{SVD}} = (2.91429,\ 3.36071,\ 3.80714,\ 4.25357,\ 4.70000)^T,} \tag{91}$$

$$\|A x_{\text{SVD}} - b_{\text{main}}\|_2 = 0.731925. \tag{92}$$

QR и SVD дают одинаковую минимальную невязку, но разные решения:

$$\|x_{\text{QR}} - x_{\text{SVD}}\|_2 = 10.4502. \tag{93}$$

При этом

$$\|x_{\text{QR}}\|_2 = 13.5525, \qquad \|x_{\text{SVD}}\|_2 = 8.62929, \tag{94}$$

что подтверждает свойство минимальной нормы SVD-решения.

Псевдообратная матрица во второй части строилась аналогично первой части по формуле $A^+ = V\Sigma^+ U^T$. Поскольку $\operatorname{rank} A = 2$, обратными заменялись только два сингулярных числа, превышающие численный

порог. Остальные сингулярные числа считались равными нулю, поэтому соответствующие элементы матрицы Σ^+ принимались нулевыми. Псевдообратная матрица:

$$A^+ = \begin{pmatrix} 0.714286 & 0.542857 & 0.371429 & 0.2 & 0.0285714 & -0.142857 & -0.314286 \\ 0.460714 & 0.35 & 0.239286 & 0.128571 & 0.0178571 & -0.0928571 & -0.203571 \\ 0.207143 & 0.157143 & 0.107143 & 0.0571429 & 0.00714286 & -0.0428571 & -0.0928571 \\ -0.0464286 & -0.0357143 & -0.025 & -0.0142857 & -0.00357143 & 0.00714286 & 0.0178571 \\ -0.3 & -0.228571 & -0.157143 & -0.0857143 & -0.0142857 & 0.0571429 & 0.128571 \end{pmatrix}. \tag{95}$$

Поскольку $r = 2 < N = 5$ и $r < 7$, обе матрицы A^+A и AA^+ являются проекторами неполного ранга и не совпадают с единичными. Получены значения

$$\left\|A^+A - I_5\right\|_F = 1.73205 = \sqrt{3}, \tag{96}$$

$$\left\|AA^+ - I_7\right\|_F = 2.23607 = \sqrt{5}, \tag{97}$$

$$\left\|(A^+A)^2 - A^+A\right\|_F = 1.94239 \cdot 10^{-14}, \tag{98}$$

$$\left\|(AA^+)^2 - AA^+\right\|_F = 3.55613 \cdot 10^{-15}, \tag{99}$$

$$\left\|(A^+A)^T - A^+A\right\|_F = 2.92081 \cdot 10^{-14}, \tag{100}$$

$$\left\|(AA^+)^T - AA^+\right\|_F = 7.01763 \cdot 10^{-15}. \tag{101}$$

Решение через псевдообратную матрицу совпало с SVD-решением:

$$\left\|A^+ b_{\text{main}} - x_{\text{SVD}}\right\|_2 = 1.07174 \cdot 10^{-13}. \tag{102}$$

Раздел Д. Большая система и анализ трудоёмкости

Таблица 3: Результаты для системы неполного ранга

N	r	T_{QR}, с	p_{QR}	T_{SVD}, с	p_{SVD}	$\|r_{QR}\|_2$	$\|r_{SVD}\|_2$
100	2	0.000517958	–	0.00610021	–	0.980487	0.980487
500	2	0.00715617	1.63152	0.0822299	1.61621	0.996020	0.996020
1000	2	0.0365528	2.35272	0.596390	2.85852	0.998005	0.998005
1500	2	0.0564230	1.07067	1.25464	1.83420	0.998669	0.998670
2000	2	0.107786	2.24995	3.47781	3.54404	0.999001	0.999001

Таблица 4: Относительные невязки для системы неполного ранга

N	$\|r_{QR}\|_2 / \|b\|_2$	$\|r_{SVD}\|_2 / \|b\|_2$
100	$3.83648 \cdot 10^{-8}$	$3.83648 \cdot 10^{-8}$
500	$1.44038 \cdot 10^{-10}$	$1.44038 \cdot 10^{-10}$
1000	$1.28091 \cdot 10^{-11}$	$1.28091 \cdot 10^{-11}$
1500	$3.10515 \cdot 10^{-12}$	$3.10515 \cdot 10^{-12}$
2000	$1.13564 \cdot 10^{-12}$	$1.13564 \cdot 10^{-12}$

Ранг матрицы остаётся равным двум при всех исследованных N. Реализованный QR-алгоритм после двух шагов обнаруживает отсутствие

новых линейно независимых столбцов и останавливается. Каждый из этих шагов обрабатывает плотную матрицу размера порядка $N \times N$, поэтому трудоёмкость составляет $O(N^2)$. Экспериментальный показатель p_{QR} колеблется из-за малого времени отдельных запусков, а на последнем переходе равен 2.24995, что согласуется с квадратичной оценкой.

Полное SVD не использует малый ранг заранее и имеет трудоёмкость $O(N^3)$. При переходе к наибольшему размеру получено $p_{SVD} = 3.54404$. Время SVD при $N = 2000$ составило 3.47781 с, тогда как QR потребовало 0.107786 с. Оба метода дают практически одинаковые невязки.

5. Выводы

В работе реализовано решение переопределённых систем в смысле наименьших квадратов с помощью QR-разложения отражениями Хаусхолдера. При решении матрица Q явно не формировалась; использовались сохранённые векторы отражений. Явная матрица Q была построена отдельно для проверки разложения без формирования и перемножения промежуточных матриц Хаусхолдера.

Для полноранговой системы QR, SVD и псевдообратная матрица дали одно и то же решение с расхождениями порядка 10^{-14}. Тестовое точное решение восстановлено с машинной точностью. Равенства $A = QR$ и $A = U\Sigma V^T$ также проверены с погрешностью порядка 10^{-14}.

Установлено, что изменение первого элемента правой части в части 1 не нарушает совместность из-за особенностей пространства столбцов заданной матрицы. Для формирования требуемой несовместной основной задачи был изменён последний элемент правой части, после чего ранг расширенной матрицы стал равен 6.

Для нечётного варианта части 2 теоретически и численно подтверждён ранг $r = 2$. QR с перестановкой столбцов и SVD дали одинаковую минимальную невязку 0.731925, но разные решения. SVD-решение имеет меньшую норму, поскольку псевдообратная матрица выбирает единственное решение МНК минимальной нормы.

Проверены свойства псевдообратной матрицы и соответствующих ортогональных проекторов.

Эксперимент подтвердил кубическую трудоёмкость QR и SVD для плотной полноранговой системы. Для специальной матрицы ранга 2 QR с ранней остановкой имеет квадратичную трудоёмкость, тогда как полное SVD сохраняет кубическую трудоёмкость.

Код

```
from dataclasses import dataclass
from time import perf_counter
import numpy as np


np.set_printoptions(suppress=True, linewidth=180, formatter={"float_kind": lambda value: f"{value:.6g}"})

# хранение результатов QR-разложения
@dataclass
class QRDecomposition:
    r: np.ndarray   # матрица R
    reflectors: list[tuple[int, np.ndarray]]    # отражение Хаусхолдера
    permutation: np.ndarray     # перестановка столбцов
    rank: int   # ранг матрицы


def separator(title: str) -> None:
    print("\n" + "=" * 100)
    print(title)
    print("=" * 100)


def fmt(value: float) -> str:
    """Представление числа с шестью значащими цифрами"""
    return f"{float(value):.6g}"


def make_exact_solution(n: int) -> np.ndarray:
    """формирование точного решения N=5 возвращает (1, 1, 8, 6, 2), год рождения Давида Гильберта — 1862"""
    if n == 5:
        return np.array([1.0, 1.0, 8.0, 6.0, 2.0])
    pattern = np.array([1.0, 8.0, 6.0, 2.0])
    x_exact = np.empty(n, dtype=np.float64)
    x_exact[0] = 1.0
    if n > 1:
        x_exact[1:] = np.resize(pattern, n - 1)
    return x_exact


def make_full_rank_matrix(n: int) -> np.ndarray:
    """формирование матрицы части 1 размера (N+2) x N"""
    if n <= 0:
        raise ValueError("N должно быть положительным")
    row_values = 10.0 / np.arange(1, n + 3, dtype=np.float64)
    a = np.repeat(row_values[:, None], n, axis=1)
    for i in range(n):
        a[i, i] = float(min(n, 10))
    return a


def make_rank_deficient_matrix(n: int) -> np.ndarray:
    """формирование матрицы части 2 для нечётного варианта: A[i,j]=i+j-2ij."""
    if n <= 0:
        raise ValueError("N должно быть положительным")
    i = np.arange(1, n + 3, dtype=np.float64)[:, None]
    j = np.arange(1, n + 1, dtype=np.float64)[None, :]
    return i + j - 2.0 * i * j


def make_part1_right_sides(a: np.ndarray, x_exact: np.ndarray, perturbation: float = 1.0) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Правые части части 1"""
    b_test = a @ x_exact
    b_first_changed = b_test.copy()
    b_first_changed[0] += perturbation
    b_main = b_test.copy()
    b_main[-1] += perturbation
    return b_test, b_first_changed, b_main


def make_part2_right_sides(a: np.ndarray, x_exact: np.ndarray, perturbation: float = 1.0) -> tuple[np.ndarray, np.ndarray]:
    """Правые части части 2"""
```

```
    b_test = a @ x_exact
    b_main = b_test.copy()
    b_main[0] += perturbation
    return b_test, b_main

# нахождения ранга
def numerical_rank(a: np.ndarray) -> int:
    return int(np.linalg.matrix_rank(a))


def householder_qr(a: np.ndarray) -> QRDecomposition:
    """QR-разложение Хаусхолдера без явного формирования Q"""
    r = np.asarray(a, dtype=np.float64).copy()
    m, n = r.shape
    reflectors = []
    tolerance = np.finfo(np.float64).eps * max(m, n) * max(1.0, np.linalg.norm(r, ord=np.inf))
    for k in range(min(m, n)):
        x = r[k:, k].copy()
        norm_x = np.linalg.norm(x)
        if norm_x <= tolerance:
            raise np.linalg.LinAlgError("Матрица не имеет полного столбцового ранга")
        alpha = -np.copysign(norm_x, x[0])
        # формирования вектора отражения
        v = x.copy()
        v[0] -= alpha
        v /= np.linalg.norm(v)
        r[k:, k:] -= 2.0 * np.outer(v, v @ r[k:, k:])
        r[k + 1:, k] = 0.0
        reflectors.append((k, v))
    r[np.abs(r) <= tolerance] = 0.0
    return QRDecomposition(r, reflectors, np.arange(n), n)


def pivoted_householder_qr(a: np.ndarray) -> QRDecomposition:
    """QR-разложение Хаусхолдера с выбором ведущего столбца: A Pi = QR"""
    r = np.asarray(a, dtype=np.float64).copy()
    m, n = r.shape
    permutation = np.arange(n)
    reflectors = []
    tolerance = np.finfo(np.float64).eps * max(m, n) * max(1.0, np.linalg.norm(r, ord=np.inf))
    for k in range(min(m, n)):
        remaining_norms = np.linalg.norm(r[k:, k:], axis=0)
        leading_position = k + int(np.argmax(remaining_norms))      # вычисления нормы для
        ↪  оставшихся столбцов
        if remaining_norms[leading_position - k] <= tolerance:
            break
        if leading_position != k:
            r[:, [k, leading_position]] = r[:, [leading_position, k]]       #если выбранный
            ↪  столбец не первый
            permutation[[k, leading_position]] = permutation[[leading_position, k]]
        x = r[k:, k].copy()
        norm_x = np.linalg.norm(x)
        alpha = -np.copysign(norm_x, x[0])
        v = x.copy()
        v[0] -= alpha
        v /= np.linalg.norm(v)
        r[k:, k:] -= 2.0 * np.outer(v, v @ r[k:, k:])
        r[k + 1:, k] = 0.0
        reflectors.append((k, v))
    r[np.abs(r) <= tolerance] = 0.0
    return QRDecomposition(r, reflectors, permutation, len(reflectors))


def apply_q_transpose(reflectors: list[tuple[int, np.ndarray]], vector: np.ndarray) ->
↪  np.ndarray:
    """вычислить Q^T vector без формирования Q"""
    result = np.asarray(vector, dtype=np.float64).copy()
    for k, v in reflectors:
        result[k:] -= 2.0 * v * np.dot(v, result[k:])
    return result


def back_substitution(upper: np.ndarray, right: np.ndarray) -> np.ndarray:
    """обратный ход"""
    n = upper.shape[0]
    solution = np.zeros(n, dtype=np.float64)
    tolerance = np.finfo(np.float64).eps * max(upper.shape) * max(1.0, np.linalg.norm(upper,
    ↪  ord=np.inf))
```

```
    for i in range(n - 1, -1, -1):
        if abs(upper[i, i]) <= tolerance:
            raise np.linalg.LinAlgError("Матрица R11 вырождена")
        solution[i] = (right[i] - np.dot(upper[i, i + 1:], solution[i + 1:])) / upper[i, i]
    return solution


def solve_least_squares_qr(decomposition: QRDecomposition, b: np.ndarray) -> tuple[np.ndarray,
↪  np.ndarray, np.ndarray]:
    """найти одно МНК-решение; свободные переменные в переставленной системе равны нулю"""
    rank = decomposition.rank
    n = decomposition.r.shape[1]
    qtb = apply_q_transpose(decomposition.reflectors, b)
    z = np.zeros(n, dtype=np.float64)
    z[:rank] = back_substitution(decomposition.r[:rank, :rank], qtb[:rank])
    x = np.zeros(n, dtype=np.float64)
    x[decomposition.permutation] = z
    return x, z, qtb


def form_q_explicitly(m: int, reflectors: list[tuple[int, np.ndarray]]) -> np.ndarray:
    """восстановить Q по столбцам без матричных перемножений и явных H_k"""
    q = np.zeros((m, m), dtype=np.float64)
    for column in range(m):
        vector = np.zeros(m, dtype=np.float64)
        vector[column] = 1.0
        for k, v in reversed(reflectors):
            vector[k:] -= 2.0 * v * np.dot(v, vector[k:])
        q[:, column] = vector
    return q

# матрица перестановки
def make_permutation_matrix(permutation: np.ndarray) -> np.ndarray:
    return np.eye(len(permutation))[:, permutation]


def solve_least_squares_svd(a: np.ndarray, b: np.ndarray) -> tuple[np.ndarray, np.ndarray,
↪  np.ndarray, np.ndarray, int]:
    """SVD"""
    u, singular_values, vt = np.linalg.svd(a, full_matrices=True)
    tolerance = np.finfo(np.float64).eps * max(a.shape) * singular_values[0]
    rank = int(np.sum(singular_values > tolerance))
    inverse_values = np.zeros_like(singular_values)
    inverse_values[:rank] = 1.0 / singular_values[:rank]
    x = vt.T @ (inverse_values * (u[:, :len(singular_values)].T @ b))
    return x, u, singular_values, vt, rank


def pseudoinverse_from_svd(u: np.ndarray, singular_values: np.ndarray, vt: np.ndarray, rank:
↪  int) -> np.ndarray:
    """псевдообратная матрица A^+=V Sigma^+ U^T"""
    inverse_values = np.zeros_like(singular_values)
    inverse_values[:rank] = 1.0 / singular_values[:rank]
    return (vt.T * inverse_values) @ u[:, :len(singular_values)].T


def make_sigma(shape: tuple[int, int], singular_values: np.ndarray) -> np.ndarray:
    """прямоугольная матрица Sigma размера m x n"""
    sigma = np.zeros(shape, dtype=np.float64)
    np.fill_diagonal(sigma, singular_values)
    return sigma

# невязка
def residual_norm(a: np.ndarray, x: np.ndarray, b: np.ndarray) -> float:
    return float(np.linalg.norm(a @ x - b))


def relative_residual_norm(a: np.ndarray, x: np.ndarray, b: np.ndarray) -> float:
    """относительная невязка ||Ax-b||_2 / ||b||_2"""
    denominator = np.linalg.norm(b)
    return float(np.linalg.norm(a @ x - b) / denominator) if denominator != 0.0 else
    ↪  float(np.linalg.norm(a @ x - b))


def empirical_order(previous_n: int | None, previous_time: float | None, n: int, current_time:
↪  float) -> str:
    """Оценка экспериментального порядка"""
```

```
    if previous_n is None or previous_time is None or previous_time <= 0.0 or current_time <= 0.0:
        return "-"
    return fmt(np.log(current_time / previous_time) / np.log(n / previous_n))


# ================================================================================================
# ЧАСТЬ 1 ЛАБОРАТОРНОЙ РАБОТЫ
# Полноранговые системы
# ================================================================================================


def run_part1() -> None:
    """Часть 1: полноранговая система, пункты 0-7"""
    separator("ЧАСТЬ 1. ПОЛНОРАНГОВЫЕ СИСТЕМЫ")
    n = 5
    a = make_full_rank_matrix(n)
    x_exact = make_exact_solution(n)
    b_test, b_first_changed, b_main = make_part1_right_sides(a, x_exact)
    rank_a = numerical_rank(a)
    rank_test = numerical_rank(np.column_stack((a, b_test)))
    rank_first_changed = numerical_rank(np.column_stack((a, b_first_changed)))
    rank_main = numerical_rank(np.column_stack((a, b_main)))

    separator("ЧАСТЬ 1. ПУНКТ 0")
    print("A =\n", a)
    print("x_* =", x_exact)
    print("b_test =", b_test)
    print("b после изменения первого элемента =", b_first_changed)
    print("b_main =", b_main)
    print("rank(A) =", rank_a)
    print("rank([A|b_test]) =", rank_test)
    print("rank([A|b с изменённым b[0]]) =", rank_first_changed)
    print("rank([A|b_main]) =", rank_main)
    print("Изменение b[0] сохраняет совместность:", rank_first_changed == rank_a)
    print("Основная задача несовместна:", rank_main > rank_a)

    decomposition = householder_qr(a)

    separator("ЧАСТЬ 1. РАЗДЕЛ А (пп. 1, 2). РЕШЕНИЕ И НЕВЯЗКА")
    x_qr_main, z_main, qtb_main = solve_least_squares_qr(decomposition, b_main)
    print("R =\n", decomposition.r)
    print("Q^T b_main =", qtb_main)
    print("x_QR =", x_qr_main)
    print("||A x_QR - b_main||_2 =", fmt(residual_norm(a, x_qr_main, b_main)))
    print("||A x_QR - b_main||_2 / ||b_main||_2 =", fmt(relative_residual_norm(a, x_qr_main, b_main)))

    separator("ЧАСТЬ 1. РАЗДЕЛ Б (п. 3). ТЕСТОВЫЙ ПРИМЕР")
    x_qr_test, z_test, qtb_test = solve_least_squares_qr(decomposition, b_test)
    print("Найденное решение =", x_qr_test)
    print("Точное решение =", x_exact)
    print("Вычисление правой части b_test = A @ x_* =", b_test)
    print("Полученная треугольная матрица R1 =\n", decomposition.r[:n, :])
    print("Правая часть треугольной системы c1 = (Q^T b_test)[0:N] =", qtb_test[:n])
    print("Треугольная система: R1 @ x = c1")
    print("||x_QR - x_*||_2 =", fmt(np.linalg.norm(x_qr_test - x_exact)))
    print("||A x_QR - b_test||_2 =", fmt(residual_norm(a, x_qr_test, b_test)))

    separator("ЧАСТЬ 1. РАЗДЕЛ В (пп. 4, 5). МАТРИЧНЫЕ РАЗЛОЖЕНИЯ")
    q = form_q_explicitly(a.shape[0], decomposition.reflectors)
    pi = np.eye(n)
    qr_product = q @ decomposition.r
    print("Q =\n", q)
    print("Pi = I, поскольку в полноранговой части перестановки столбцов не выполняются:\n", pi)
    print("R =\n", decomposition.r)
    print("A Pi =\n", a @ pi)
    print("Q R =\n", qr_product)
    print("||Q^T Q - I||_F =", fmt(np.linalg.norm(q.T @ q - np.eye(n + 2), ord="fro")))
    print("||A - QR||_F =", fmt(np.linalg.norm(a - qr_product, ord="fro")))
    print("Проверка A = QR:", np.allclose(a, qr_product))

    separator("ЧАСТЬ 1. РАЗДЕЛ Г (пп. 6, 7). SVD И ПСЕВДООБРАТНАЯ МАТРИЦА")
    x_svd_main, u, singular_values, vt, rank_svd = solve_least_squares_svd(a, b_main)
```

```
    x_svd_test, u_test, singular_values_test, vt_test, rank_svd_test = solve_least_squares_svd(a, b_test)
    v = vt.T
    sigma = make_sigma(a.shape, singular_values)
    print("U =\n", u)
    print("V =\n", v)
    print("Sigma =\n", sigma)
    print("V^T =\n", vt)
    print("Сингулярные числа =", singular_values)
    print("Ранг по SVD =", rank_svd)
    print("||U^T U - I||_F =", fmt(np.linalg.norm(u.T @ u - np.eye(u.shape[0]), ord="fro")))
    print("||V^T V - I||_F =", fmt(np.linalg.norm(v.T @ v - np.eye(v.shape[0]), ord="fro")))
    print("U^T A V =\n", u.T @ a @ v)
    print("||U^T A V - Sigma||_F =", fmt(np.linalg.norm(u.T @ a @ v - sigma, ord="fro")))
    print("||A - U Sigma V^T||_F =", fmt(np.linalg.norm(a - u @ sigma @ vt, ord="fro")))
    print("SVD-решение тестовой задачи =", x_svd_test)
    print("Ошибка тестового решения =", fmt(np.linalg.norm(x_svd_test - x_exact)))
    print("SVD-решение основной задачи =", x_svd_main)
    print("Невязка основной задачи =", fmt(residual_norm(a, x_svd_main, b_main)))
    print("||x_QR - x_SVD||_2 =", fmt(np.linalg.norm(x_qr_main - x_svd_main)))

    a_plus = pseudoinverse_from_svd(u, singular_values, vt, rank_svd)
    a_plus_a = a_plus @ a
    a_a_plus = a @ a_plus
    x_pinv = a_plus @ b_main
    print("Способ построения: A^+ = V Sigma^+ U^T; ненулевые сингулярные числа заменяются обратными.")
    print("A^+ =\n", a_plus)
    print("A^+ A =\n", a_plus_a)
    print("A A^+ =\n", a_a_plus)
    print("||A^+ A - I_N||_F =", fmt(np.linalg.norm(a_plus_a - np.eye(n), ord="fro")))
    print("||A A^+ - I_(N+2)||_F =", fmt(np.linalg.norm(a_a_plus - np.eye(n + 2), ord="fro")))
    print("||(A^+A)^2 - A^+A||_F =", fmt(np.linalg.norm(a_plus_a @ a_plus_a - a_plus_a, ord="fro")))
    print("||(AA^+)^2 - AA^+||_F =", fmt(np.linalg.norm(a_a_plus @ a_a_plus - a_a_plus, ord="fro")))
    print("||(A^+A)^T - A^+A||_F =", fmt(np.linalg.norm(a_plus_a.T - a_plus_a, ord="fro")))
    print("||(AA^+)^T - AA^+||_F =", fmt(np.linalg.norm(a_a_plus.T - a_a_plus, ord="fro")))
    print("x = A^+ b_main =", x_pinv)
    print("||x_pinv - x_SVD||_2 =", fmt(np.linalg.norm(x_pinv - x_svd_main)))


def benchmark_part1(n_values: list[int]) -> None:
    separator("ЧАСТЬ 1. РАЗДЕЛ Д (п. 8). БОЛЬШАЯ СИСТЕМА И АНАЛИЗ ТРУДОЁМКОСТИ")
    print(f"{'N':>7} {'QR, c':>12} {'p_QR':>8} {'SVD, c':>12} {'p_SVD':>8} {'невязка QR':>14} {'отн. невязка':>14} {'невязка SVD':>14} {'отн. невязка':>14}")
    previous_n = None
    previous_qr_time = None
    previous_svd_time = None
    for n in n_values:
        a = make_full_rank_matrix(n)
        x_exact = make_exact_solution(n)
        b_test, b_first_changed, b_main = make_part1_right_sides(a, x_exact)
        qr_times = []
        for repeat in range(TIMING_REPEATS):
            start = perf_counter()
            decomposition = householder_qr(a)
            x_qr, z_qr, qtb = solve_least_squares_qr(decomposition, b_main)
            qr_times.append(perf_counter() - start)
        qr_time = float(np.median(qr_times))
        svd_times = []
        for repeat in range(TIMING_REPEATS):
            start = perf_counter()
            x_svd, u, singular_values, vt, rank_svd = solve_least_squares_svd(a, b_main)
            svd_times.append(perf_counter() - start)
        svd_time = float(np.median(svd_times))
        qr_order = empirical_order(previous_n, previous_qr_time, n, qr_time)
        svd_order = empirical_order(previous_n, previous_svd_time, n, svd_time)
        print(f"{n:7d} {qr_time:12.6g} {qr_order:>8} {svd_time:12.6g} {svd_order:>8} {residual_norm(a, x_qr, b_main):14.6g} {relative_residual_norm(a, x_qr, b_main):14.6g} {residual_norm(a, x_svd, b_main):14.6g} {relative_residual_norm(a, x_svd, b_main):14.6g}")
        previous_n = n
        previous_qr_time = qr_time
        previous_svd_time = svd_time
        if max(qr_time, svd_time) >= MAX_TIME_SECONDS:
            print("Достигнуто ограничение в один час; дальнейшее увеличение N остановлено.")
            break
```

```
    print("Теоретически для плотной полноранговой матрицы QR и SVD имеют кубическую трудоёмкость O(N^3); эмпирические p оцениваются по измеренному времени.")


# ============================================================================================== ======
# НАЧАЛО ЧАСТИ 2 ЛАБОРАТОРНОЙ РАБОТЫ
# Системы неполного ранга, нечётный вариант
# ============================================================================================== ======


def run_part2() -> None:
    """Часть 2: система неполного ранга нечётного варианта, пункты 1-7."""
    separator("ЧАСТЬ 2. СИСТЕМЫ НЕПОЛНОГО РАНГА. НЕЧЁТНЫЙ ВАРИАНТ")
    n = 5
    a = make_rank_deficient_matrix(n)
    x_exact = make_exact_solution(n)
    b_test, b_main = make_part2_right_sides(a, x_exact)
    rank_a = numerical_rank(a)
    rank_test = numerical_rank(np.column_stack((a, b_test)))
    rank_main = numerical_rank(np.column_stack((a, b_main)))
    print("A =\n", a)
    print("x_* =", x_exact)
    print("b_test =", b_test)
    print("b_main =", b_main)
    print("rank(A) =", rank_a)
    print("rank([A|b_test]) =", rank_test)
    print("rank([A|b_main]) =", rank_main)
    print("Тестовая система совместна:", rank_test == rank_a)
    print("Основная система несовместна:", rank_main > rank_a)

    decomposition = pivoted_householder_qr(a)

    separator("ЧАСТЬ 2. РАЗДЕЛ А (пп. 1, 2). РЕШЕНИЕ И НЕВЯЗКА")
    x_qr_main, z_main, qtb_main = solve_least_squares_qr(decomposition, b_main)
    print("Перестановка столбцов (нумерация с 1) =", decomposition.permutation + 1)
    print("Ранг, найденный QR-разложением =", decomposition.rank)
    print("R =\n", decomposition.r)
    print("Q^T b_main =", qtb_main)
    print("z =", z_main)
    print("Одно из МНК-решений x_QR =", x_qr_main)
    print("||A x_QR - b_main||_2 =", fmt(residual_norm(a, x_qr_main, b_main)))
    print("||A x_QR - b_main||_2 / ||b_main||_2 =", fmt(relative_residual_norm(a, x_qr_main, b_main)))

    separator("ЧАСТЬ 2. РАЗДЕЛ Б (п. 3). ТЕСТОВЫЙ ПРИМЕР")
    x_qr_test, z_test, qtb_test = solve_least_squares_qr(decomposition, b_test)
    print("Заданное решение x_* =", x_exact)
    print("Найденное QR-решение =", x_qr_test)
    print("Вычисление правой части b_test = A @ x_* =", b_test)
    print("Главная треугольная матрица R11 =\n", decomposition.r[:decomposition.rank, :decomposition.rank])
    print("Правая часть треугольной системы c1 = (Q^T b_test)[0:r] =", qtb_test[:decomposition.rank])
    print("Треугольная система: R11 @ z1 = c1; свободные компоненты z2 приняты равными нулю.")
    print("||A x_QR - b_test||_2 =", fmt(residual_norm(a, x_qr_test, b_test)))
    print("||x_QR - x_*||_2 =", fmt(np.linalg.norm(x_qr_test - x_exact)))
    print("Различие решений допустимо: при rank(A) < N совместная система имеет бесконечно много решений.")

    separator("ЧАСТЬ 2. РАЗДЕЛ В (пп. 4, 5). МАТРИЧНЫЕ РАЗЛОЖЕНИЯ")
    q = form_q_explicitly(a.shape[0], decomposition.reflectors)
    pi = make_permutation_matrix(decomposition.permutation)
    a_pi = a @ pi
    qr_product = q @ decomposition.r
    print("Q =\n", q)
    print("Pi =\n", pi)
    print("A Pi =\n", a_pi)
    print("Q R =\n", qr_product)
    print("||Q^T Q - I||_F =", fmt(np.linalg.norm(q.T @ q - np.eye(n + 2), ord="fro")))
    print("||A Pi - QR||_F =", fmt(np.linalg.norm(a_pi - qr_product, ord="fro")))
    print("Проверка A Pi = QR:", np.allclose(a_pi, qr_product))

    separator("ЧАСТЬ 2. РАЗДЕЛ Г (пп. 6, 7). SVD И ПСЕВДООБРАТНАЯ МАТРИЦА")
    x_svd_main, u, singular_values, vt, rank_svd = solve_least_squares_svd(a, b_main)
    x_svd_test, u_test, singular_values_test, vt_test, rank_svd_test = solve_least_squares_svd(a, b_test)
```

```
    v = vt.T
    sigma = make_sigma(a.shape, singular_values)
    print("U =\n", u)
    print("V =\n", v)
    print("Sigma =\n", sigma)
    print("V^T =\n", vt)
    print("Сингулярные числа =", singular_values)
    print("Ранг по SVD =", rank_svd)
    print("||U^T U - I||_F =", fmt(np.linalg.norm(u.T @ u - np.eye(u.shape[0]), ord="fro")))
    print("||V^T V - I||_F =", fmt(np.linalg.norm(v.T @ v - np.eye(v.shape[0]), ord="fro")))
    print("U^T A V =\n", u.T @ a @ v)
    print("||U^T A V - Sigma||_F =", fmt(np.linalg.norm(u.T @ a @ v - sigma, ord="fro")))
    print("||A - U Sigma V^T||_F =", fmt(np.linalg.norm(a - u @ sigma @ vt, ord="fro")))
    print("SVD-решение тестовой задачи =", x_svd_test)
    print("Невязка тестовой задачи =", fmt(residual_norm(a, x_svd_test, b_test)))
    print("SVD-решение основной задачи =", x_svd_main)
    print("Невязка основной задачи =", fmt(residual_norm(a, x_svd_main, b_main)))
    print("||x_QR - x_SVD||_2 =", fmt(np.linalg.norm(x_qr_main - x_svd_main)))
    print("||x_QR||_2 =", fmt(np.linalg.norm(x_qr_main)))
    print("||x_SVD||_2 =", fmt(np.linalg.norm(x_svd_main)))

    a_plus = pseudoinverse_from_svd(u, singular_values, vt, rank_svd)
    a_plus_a = a_plus @ a
    a_a_plus = a @ a_plus
    x_pinv = a_plus @ b_main
    print("Способ построения: A^+ = V Sigma^+ U^T; только ненулевые сингулярные числа
    ↪  заменяются обратными.")
    print("A^+ =\n", a_plus)
    print("A^+ A =\n", a_plus_a)
    print("A A^+ =\n", a_a_plus)
    print("||A^+ A - I_N||_F =", fmt(np.linalg.norm(a_plus_a - np.eye(n), ord="fro")))
    print("||A A^+ - I_(N+2)||_F =", fmt(np.linalg.norm(a_a_plus - np.eye(n + 2), ord="fro")))
    print("||(A^+A)^2 - A^+A||_F =", fmt(np.linalg.norm(a_plus_a @ a_plus_a - a_plus_a,
    ↪  ord="fro")))
    print("||(AA^+)^2 - AA^+||_F =", fmt(np.linalg.norm(a_a_plus @ a_a_plus - a_a_plus,
    ↪  ord="fro")))
    print("||(A^+A)^T - A^+A||_F =", fmt(np.linalg.norm(a_plus_a.T - a_plus_a, ord="fro")))
    print("||(AA^+)^T - AA^+||_F =", fmt(np.linalg.norm(a_a_plus.T - a_a_plus, ord="fro")))
    print("x = A^+ b_main =", x_pinv)
    print("||x_pinv - x_SVD||_2 =", fmt(np.linalg.norm(x_pinv - x_svd_main)))


def benchmark_part2(n_values: list[int]) -> None:
    separator("ЧАСТЬ 2. РАЗДЕЛ Д (п. 8). БОЛЬШАЯ СИСТЕМА И АНАЛИЗ ТРУДОЁМКОСТИ")
    print(f"{'N':>7} {'rank':>6} {'QR, c':>12} {'p_QR':>8} {'SVD, c':>12} {'p_SVD':>8}
    ↪  {'невязка QR':>14} {'отн. невязка':>14} {'невязка SVD':>14} {'отн. невязка':>14}")
    previous_n = None
    previous_qr_time = None
    previous_svd_time = None
    for n in n_values:
        a = make_rank_deficient_matrix(n)
        x_exact = make_exact_solution(n)
        b_test, b_main = make_part2_right_sides(a, x_exact)
        qr_times = []
        for repeat in range(TIMING_REPEATS):
            start = perf_counter()
            decomposition = pivoted_householder_qr(a)
            x_qr, z_qr, qtb = solve_least_squares_qr(decomposition, b_main)
            qr_times.append(perf_counter() - start)
        qr_time = float(np.median(qr_times))
        svd_times = []
        for repeat in range(TIMING_REPEATS):
            start = perf_counter()
            x_svd, u, singular_values, vt, rank_svd = solve_least_squares_svd(a, b_main)
            svd_times.append(perf_counter() - start)
        svd_time = float(np.median(svd_times))
        qr_order = empirical_order(previous_n, previous_qr_time, n, qr_time)
        svd_order = empirical_order(previous_n, previous_svd_time, n, svd_time)
        print(f"{n:7d} {rank_svd:6d} {qr_time:12.6g} {qr_order:>8} {svd_time:12.6g}
        ↪  {svd_order:>8} {residual_norm(a, x_qr, b_main):14.6g} {relative_residual_norm(a,
        ↪  x_qr, b_main):14.6g} {residual_norm(a, x_svd, b_main):14.6g}
        ↪  {relative_residual_norm(a, x_svd, b_main):14.6g}")
        previous_n = n
        previous_qr_time = qr_time
        previous_svd_time = svd_time
        if max(qr_time, svd_time) >= MAX_TIME_SECONDS:
            print("Достигнуто ограничение в один час; дальнейшее увеличение N остановлено.")
            break
```

```
    print("Для этой матрицы rank(A)=2: QR с ранней остановкой обрабатывает два ведущих столбца, а полное SVD остаётся существенно более трудоёмким.")


# Требуемые размеры из раздела Д
LARGE_N_VALUES = [100, 500, 1000]
MAX_TIME_SECONDS = 3600.0
TIMING_REPEATS = 3


# ЗАПУСК ЧАСТИ 1
run_part1()
benchmark_part1(LARGE_N_VALUES)

# ЗАПУСК ЧАСТИ 2
run_part2()
benchmark_part2(LARGE_N_VALUES)
```